Cardiff University

Department of Computer Science

GPS Safety Warning App for Minor-to-Major Road Merges Using OpenStreetMap and Mobile Location Tracking

Ghala Alhawas

Supervisor: Crispin Cooper

A report submitted in partial fulfilment of the requirements of
Cardiff University for the degree of
Bachelor of Science in *Computer Science*

May 6, 2026

Declaration

I, Ghala Alhawas, of the Department of Computer Science, Cardiff University, confirm that this is my own work and figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. I understand that failure to do so will be considered plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.
I give consent to a copy of my report being shared with future students as an exemplar.
I give consent for my work to be made available more widely to members of Cardiff University and the public with interest in teaching, learning and research.

Ghala Alhawas
May 6, 2026

Abstract

This report presents the design, implementation, and evaluation of a GPS hazard-warning system for drivers in unfamiliar road environments. The scope is intentionally narrow: warning drivers before high-risk minor-to-major merge points. Rather than replicating full navigation functionality, the system targets early, context-aware alerts at a specific infrastructure risk.

The solution combines an offline geospatial pipeline with a mobile runtime application. The Python layer uses OpenStreetMap data and OSMnx to extract the Oxford road network, detect junctions, score merge risk using class-mismatch and approach-angle weighting, and generate curve-aware displaced pre-junction warning points along actual road geometry. The mobile layer, implemented in React Native with Expo, performs live location tracking, R-tree spatial candidate filtering, bearing-aware approach checks, cooldown de-duplication, and warning presentation with visual and haptic feedback.

A dedicated developer mode is built into the app to support repeatable in-application testing without requiring physical travel to hazard locations. Developer mode provides simulated GPS placement near high-danger junctions, fake-GPS toggling with real-GPS restoration, a map jump panel, and state snapshot and recovery logic. This capability was essential for validating warning behavior during development and report evidence capture.

Using the current implementation outputs, 2,258 junctions were identified, 455 hazard points were generated in GeoJSON, and compact mobile bundles were produced containing 274 full-dataset records and a 50-point sample subset. The mobile app delivers visual and haptic warning feedback and includes the developer-mode workflow for repeatable scenario validation.

The results demonstrate that an offline-first, lightweight architecture can deliver robust merge-specific warning behavior without cloud dependencies. Principal limitations are limited field-trial evidence, sensitivity to GPS drift in dense urban settings, and the need for multi-region validation.

Keywords: road safety, OpenStreetMap, mobile geospatial systems, hazard detection, GPS warning, React Native, developer mode testing

Acknowledgements

I would like to thank my supervisor, Crispin Cooper, for his guidance, practical feedback, and support throughout this Final Year Project.

I would also like to thank the Department of Computer Science at Cardiff University for providing the academic structure, facilities, and resources needed to complete this work.

Finally, I am grateful to my family and friends for their encouragement during planning, implementation, testing, and report writing.

Contents

List of Figures

List of Tables

List of Abbreviations

API	Application Programming Interface
CSV	Comma-Separated Values
DSE	Display Screen Equipment
GDPR	General Data Protection Regulation
GeoJSON	Geographic JavaScript Object Notation
GPS	Global Positioning System
HSE	Health and Safety Executive
JSON	JavaScript Object Notation
OSM	OpenStreetMap
R-tree	Spatial tree index used for fast geographic lookups
UI	User Interface
UK	United Kingdom
UX	User Experience

Chapter 1

Introduction

1.1 Background

Driving in unfamiliar countries introduces avoidable risk at local road merges, especially where a narrow minor road joins a faster major road. In these situations, drivers can make late decisions, approach at the wrong speed, or position incorrectly before entering the main carriageway. Existing navigation tools prioritize route guidance and traffic conditions but do not explicitly model this merge-specific hazard context.

This project evaluates whether a focused mobile warning system can be implemented using open geospatial data and lightweight on-device computation, without cloud inference or complex backend infrastructure. The implementation deliberately concentrates on one scenario: minor-to-major merge approach warnings.

1.2 Problem Statement

The technical problem addressed in this report is the absence of a practical, offline-first warning workflow that combines:

- reliable extraction of risky merge candidates from OpenStreetMap road topology;
- geospatial hazard-point generation with advance warning placement;
- real-time mobile detection using position and approach direction; and
- low-latency warning delivery suitable for in-motion use.

Without this integration, the workflow remains fragmented: map data is available but not translated into actionable merge-risk warnings, while mobile positioning is available but not semantically linked to hazard context.

1.3 Aim and Objectives

Aim: To design and implement a focused GPS safety application that warns drivers when approaching high-risk minor-to-major road merges using OpenStreetMap-derived hazard data.

Objectives:

1. Build a reproducible Python pipeline to download road-network data, detect junctions, and generate hazard points.
2. Prioritize minor-to-major merge detection using road-class mismatch and junction geometry.
3. Export mobile-friendly hazard datasets suitable for offline runtime use.
4. Implement a React Native (Expo) app with live GPS tracking, map display, and in-app warnings.
5. Use spatial indexing and distance filtering to keep runtime checks efficient on mobile devices.
6. Implement bearing-aware warning logic and cooldown protection to reduce irrelevant alerts.
7. Provide developer-mode controls for repeatable in-app validation of warning behavior.

1.4 Research Questions and Success Criteria

The project is framed by the following research questions:

- **RQ1:** Can OpenStreetMap network data be transformed into a useful set of merge-hazard warning points with a reproducible processing pipeline?
- **RQ2:** Can a mobile app trigger timely and context-aware warnings using only local hazard data and live GPS updates?
- **RQ3:** Can this warning workflow be kept simple enough for practical deployment and demonstration within final-year project constraints?

Success was assessed against explicit criteria:

1. junction detection and hazard generation execute end-to-end without manual data correction;

2. generated hazard points represent valid pre-junction warning positions;
3. mobile runtime can load hazard data and run warning checks continuously;
4. warning decisions apply distance, score, bearing, and cooldown constraints consistently;
5. the app remains usable during real-device testing and Expo deployment;
6. developer-mode controls support repeatable validation of warning behavior.

Evidence for these criteria is presented in Chapter 3, Chapter 4, and the appendices.

1.5 Scope

The scope is intentionally narrow and implementation-driven. It covers one city dataset (Oxford), one primary warning scenario (minor-to-major merges), and one mobile prototype. It excludes turn-by-turn navigation, cloud services, user accounts, and production-scale telemetry pipelines.

1.6 Solution Overview

The solution has two connected layers:

- **Offline geospatial pipeline (Python):** download network data, detect junctions, score risk, generate displaced warning points, and export compact mobile JSON.
- **Mobile runtime (React Native + Expo):** track location, query nearby hazards via R-tree filtering, check direction and cooldown constraints, then present warning feedback.

This separation places computationally intensive geospatial processing offline and preserves low-latency runtime checks on mobile.

1.7 Contributions

The principal contributions are:

- a complete Oxford-focused hazard-generation pipeline based on OpenStreetMap data;

- a merge-focused danger-scoring approach that prioritizes class mismatch and angle context;
- curve-aware warning-point displacement to provide pre-junction warning distance;
- a working mobile warning app with offline cache, spatial indexing, and bearing-aware checks;
- developer-mode controls for practical, repeatable runtime validation.

1.8 Report Structure

Chapter 2 reviews literature on road-safety context, open geospatial data, mobile location tracking, and spatial-query performance.

Chapter 3 presents the implementation methodology, architecture, data-processing workflow, and verification strategy, with representative code excerpts from both Python and mobile components.

Chapter 4 reports pipeline outputs, runtime behavior, and objective attainment using the current project datasets.

Chapter 5 discusses interpretation, limitations, and validity considerations.

Chapter 6 concludes the work and proposes future engineering directions, and Chapter 7 presents personal technical reflection.

Chapter 2

Literature Review

2.1 Road-Safety Context for Junction Risk

Road traffic incidents remain a major global safety concern, with junction behavior and approach decisions playing a central role in collision risk (World Health Organization, 2023). UK transport statistics also show the practical importance of junction-related and maneuver-related safety interventions (Department for Transport, 2025).

The central context for this project is therefore merge-specific risk: drivers on minor roads entering faster major roads with limited anticipation time.

2.2 Open Geospatial Data for Transport Analysis

OpenStreetMap (OSM) provides a widely used, community-maintained map dataset suitable for research and applied geospatial systems (OpenStreetMap Foundation, 2026). OSMnx enables direct conversion of OSM data into analyzable graph structures, which makes it suitable for road-network extraction, node-level analysis, and reproducible map-based pipelines (Boeing, 2017; OSMnx Contributors, 2026).

OSM and OSMnx are adopted because they provide broad accessibility and sufficient structural detail (nodes, edges, road classes, and geometry) for merge-hazard detection.

2.3 Hazard Modeling from Junction Geometry and Road Class

Map-derived hazard modeling typically balances two constraints: semantic relevance (is the location meaningfully risky?) and computational efficiency (can warnings be evaluated within runtime constraints). A robust compromise is rule-based scoring using consistently available graph features such as road-class mismatch and approach geometry.

The present work follows this approach by prioritizing minor-to-major class mismatch and acute-angle geometry signals. The objective is not full accident prediction, but a reproducible risk proxy suitable for mobile deployment.

2.4 Mobile Location Tracking and Runtime Trade-offs

Mobile location systems face accuracy-energy trade-offs: higher sampling and precision improve positional confidence but increase battery use. Expo Location provides configurable update intervals and accuracy levels suitable for this trade-off in React Native applications (Expo, 2026; Meta Open Source, 2026).

The design implication is clear: warning logic should tolerate moderate GPS noise while avoiding excessive polling overhead.

2.5 Spatial Indexing for Real-Time Nearby Search

A naive scan over all hazards on every GPS update does not scale well as datasets grow. Spatial indexing is therefore a standard optimization in geospatial runtime systems. R-tree structures are commonly used to reduce candidate sets before exact distance checks (Guttman, 1984; Agafonkin, 2026).

Accordingly, this project uses an R-tree (rbush) for bounding-box prefiltering, followed by precise Haversine distance evaluation.

2.6 Map-Centric Mobile Interaction

For this project context, interaction quality is defined by low-friction visibility of location and hazards, clear warning salience, and minimal cognitive overhead while moving. React Native Maps is widely used for map rendering in mobile prototypes and supports marker overlays and location visualization in this interaction style (react-native-maps Maintainers, 2026).

The interface therefore prioritizes map clarity, concise warning text, and low-friction controls.

2.7 Similar Existing Systems

The project was compared against mainstream navigation applications that users already rely on:

- **Google Maps:** strong route guidance and traffic awareness, but no explicit minor-to-major merge hazard model in user-facing workflow (Google, 2026).
- **Waze:** strong community event reporting and incident alerts, but hazard signaling remains event-centric rather than topology-driven for merge risk (Waze, 2026).

These platforms are broader and more mature; however, they do not directly target the merge-warning behavior implemented in this project.

2.8 Comparative Analysis and Practical Gap

The review indicates a practical gap between two extremes:

- full-scale navigation platforms with broad features but limited merge-specific warning logic;
- custom research prototypes that can model risk but are often difficult to deploy on consumer mobile workflows.

This project addresses that gap through a narrow objective: a deployable mobile prototype focused on one hazard scenario and supported by a reproducible data pipeline.

2.9 Design Principles Derived from Literature

Five implementation principles were derived from the reviewed sources:

1. **Scenario focus:** restrict the warning model to minor-to-major merge behavior to keep scope reliable.
2. **Data reproducibility:** use open OSM data and scriptable processing for traceable outputs (OpenStreetMap Foundation, 2026; OSMnx Contributors, 2026).
3. **Runtime efficiency:** combine spatial prefiltering with exact distance checks (Agafonkin, 2026; Guttman, 1984).
4. **Mobile practicality:** tune GPS updates for balanced performance and battery impact (Expo, 2026).
5. **Clear alerting:** keep warning presentation simple and immediate in a map-first interface (react-native-maps Maintainers, 2026).

2.10 Synthesis, Research Gap, and Chapter Summary

The literature supports an engineering path that is open-data-driven, computationally lightweight, and scenario-specific. However, there is limited publicly documented work that combines: (1) scriptable merge-hazard generation from OSM, (2) curve-aware warning-point displacement, and (3) offline-first mobile warning logic within one undergraduate-scale implementation.

The following chapter translates these literature-derived principles into the implemented architecture, code-level decisions, and verification strategy.

Chapter 3

Methodology

3.1 Methodological Framework

The project follows an iterative engineering workflow across two connected layers:

1. an offline Python geospatial pipeline for hazard-data generation; and
2. a mobile runtime layer for warning detection and delivery.

Each iteration included requirement refinement, implementation, execution, output inspection, and supervisor-aligned scope reduction. The final methodology prioritizes reliability, reproducibility, and stable runtime behavior over feature breadth.

3.2 Development Timeline (Weeks 3–39)

The implementation progressed in phased blocks from data exploration to mobile integration and report evidence capture.

3.3 System Architecture

The architecture is modular and separates offline processing from mobile runtime logic.

Pipeline Layer (Python)

- **OSM ingestion** (`01_download_osm_data.py`): download and cache Oxford road-network graph as GraphML.
- **Junction detection** (`02_detect_junctions.py`): identify nodes where `street_count` is three or more and classify by type.

Table 3.1: Phase-level development plan and milestone intent

Weeks	Phase	Primary Outcomes
4–7	Data Foundation	OSM download, network parsing, junction detection, and preliminary scoring prototypes.
8–12	Hazard Modeling	Enhanced danger scoring, minor-to-major merge filtering, and curve-aware point displacement.
13–16	Mobile Runtime	React Native map app, GPS subscription, warning banner, bearing and cooldown checks.
17–20	Runtime Tuning	Offline caching, R-tree indexing, export simplification, and warning-threshold refinement.
21–28	Evaluation and Reporting	Developer-mode validation controls, dataset and result summaries, and dissertation completion.

- **Hazard scoring** (`03_generate_hazard_points.py`): compute danger score from class mismatch and approach-angle context.
- **Hazard-point generation**: create pre-junction warning points displaced 75 m along actual road geometry.
- **Mobile export** (`04_export_hazard_data.py`): convert GeoJSON output into compact JSON files for app runtime.

Mobile Layer (React Native + Expo)

- **Location service**: requests permission, obtains current position, and watches periodic GPS updates.
- **Hazard service** (`hazardService.js`): initializes hazard data from bundled or cached JSON, and serves nearby hazards.
- **Spatial search**: R-tree bounding-box prefilter followed by Haversine distance check.
- **Warning logic**: score threshold, direction filter, and cooldown gate applied in sequence.
- **UI feedback**: map markers, warning banner (`WarningBanner.js`), and vibration.
- **Developer controls**: test-panel actions for simulated hazard approach validation without requiring physical travel.

3.4 Data Model and Export Strategy

The offline pipeline produces two main processed datasets:

- `oxford_uk_hazard_points.geojson`: full spatial hazard features for analysis.
- `oxford_uk_hazard_mobile.json`: compact runtime records for the mobile app.

For rapid testing on mobile, an additional subset is produced:

- `oxford_uk_hazard_mobile_sample50.json`: top 50 highest-scoring points for fast validation.

Each exported record contains the warning-point latitude and longitude (displaced from the junction), the junction center coordinates, warning bearing, danger score, junction type, merge type, and road-class metadata. The compact mobile record format is shown in Appendix B.

3.5 Requirements and Acceptance Criteria

Functional and runtime requirements were translated into measurable checks before final integration.

Table 3.2: Core requirements and acceptance criteria

ID	Requirement	Acceptance Criterion
R1	Reproducible junction extraction	Oxford network processing consistently produces junction dataset from cached or downloaded OSM data.
R2	Merge-focused hazard generation	Hazard points are produced only when minor-to-major class differences satisfy configured thresholds.
R3	Curve-aware warning placement	Generated warning points are displaced before the junction along road geometry where possible.
R4	Efficient runtime lookup	Nearby-hazard detection executes on each GPS update without scanning all records.
R5	Context-aware warning decisions	Warnings require score, proximity, direction, and cooldown checks to pass.
R6	Practical testing workflow	Developer mode enables repeatable simulated hazard approach testing directly inside the app.

3.6 Engineering Decisions and Trade-offs

Key architectural decisions were made by balancing safety relevance, implementation complexity, and delivery constraints:

- **Offline-first pipeline:** heavy geospatial computation is done before runtime, reducing on-device complexity.
- **Focused hazard model:** only minor-to-major merges are targeted, improving interpretability while reducing feature breadth.
- **Balanced location sampling:** lower battery impact at the cost of less frequent position updates.
- **R-tree + Haversine strategy:** efficient candidate filtering plus accurate distance checks.
- **Simple warning UI:** fast, clear alerts with limited interaction depth while driving.
- **In-app developer mode:** simulated GPS testing avoids dependency on real-world travel during development.

3.7 Spatial Utility Functions

The shared geospatial utility module (`src/utils/geo.js`) provides three functions used across the mobile runtime: Haversine distance, forward bearing, and angle difference. These are used by the hazard service for proximity checks and direction-match decisions.

Listing 3.1 shows all three functions as implemented.

```
const EARTH_RADIUS_M = 6371000;

function haversine(lat1, lon1, lat2, lon2) {
  const toRad = (deg) => (deg * Math.PI) / 180;
  const dLat = toRad(lat2 - lat1);
  const dLon = toRad(lon2 - lon1);
  const a =
    Math.sin(dLat / 2) ** 2 +
    Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) *
    Math.sin(dLon / 2) ** 2;
  return 2 * EARTH_RADIUS_M * Math.asin(Math.sqrt(a));
}

```

```
function bearingBetween(lat1, lon1, lat2, lon2) {
  const toRad = (d) => (d * Math.PI) / 180;
  const toDeg = (r) => (r * 180) / Math.PI;
  const dLon = toRad(lon2 - lon1);
  const y = Math.sin(dLon) * Math.cos(toRad(lat2));
  const x =
    Math.cos(toRad(lat1)) * Math.sin(toRad(lat2)) -
    Math.sin(toRad(lat1)) * Math.cos(toRad(lat2)) * Math.cos(dLon);
  return (toDeg(Math.atan2(y, x)) + 360) % 360;
}

function angleDiff(b1, b2) {
  const diff = Math.abs(b1 - b2) % 360;
  return Math.min(diff, 360 - diff);
}
```

Code Listing 3.1. Geospatial utility functions: Haversine distance, bearing, and angle difference

The `haversine` function computes great-circle distance, which is used in the hazard-service proximity filter. `bearingBetween` computes the forward heading between two GPS positions, used when deriving the user's travel direction from successive GPS updates. `angleDiff` normalizes the difference between two compass headings to a 0–180 range, enabling symmetric bearing comparison for approach-direction checks.

A mirror of these distance and bearing formulas is used in the Python pipeline (`src/03_generate_hazard_points.py`) to compute warning-point displacements during offline hazard generation.

3.8 Python Pipeline Implementation

Junction Detection

Script `02_detect_junctions.py` identifies all road nodes in the Oxford network with three or more connected streets (`street_count >= 3`). These become the candidate junction pool. Listing 3.2 shows the junction identification and classification logic.

```
def identify_junctions(nodes, min_streets=3):
    junctions = nodes[nodes['street_count'] >= min_streets].copy()
    return junctions

```

```
def classify_junctions(junctions):
    def get_junction_type(street_count):
        if street_count == 3:
            return "T-junction"
        elif street_count == 4:
            return "Crossroads"
        elif street_count == 5:
            return "5-way"
        else:
            return f"{street_count}-way"
    junctions['junction_type'] = junctions[
        'street_count'].apply(get_junction_type)
    return junctions
```

Code Listing 3.2. Junction identification and type classification from OSM network

The result is a GeoJSON containing 2,258 classified junctions for the Oxford road network.

Curve-Aware Warning-Point Displacement

A core pipeline design decision was to place warning points along the actual road geometry rather than at a fixed straight-line offset from the junction. This preserves road-curve context and ensures displaced points remain on the road surface. Listing 3.3 shows the `displace_along_geometry` function.

```
def displace_along_geometry(junction_lat, junction_lon,
                            edge_geometry, junction_is_start,
                            distance_m):
    if edge_geometry is None or edge_geometry.is_empty:
        return None

    coords = list(edge_geometry.coords)  # (lon, lat) pairs
    if len(coords) < 2:
        return None

    jpt = (junction_lon, junction_lat)
    start_dist = math.hypot(
        coords[0][0] - jpt[0], coords[0][1] - jpt[1])
    end_dist   = math.hypot(
```

```
        coords[-1][0] - jpt[0], coords[-1][1] - jpt[1])

    if end_dist < start_dist:
        coords = list(reversed(coords))

    walked = 0.0
    prev_lon, prev_lat = coords[0]

    for lon2, lat2 in coords[1:]:
        seg_len = haversine_distance(
            prev_lat, prev_lon, lat2, lon2)
        if walked + seg_len >= distance_m:
            remaining = distance_m - walked
            frac = remaining / seg_len if seg_len > 0 else 0
            new_lon = prev_lon + frac * (lon2 - prev_lon)
            new_lat = prev_lat + frac * (lat2 - prev_lat)
            approach_bearing = bearing_between(
                new_lat, new_lon, junction_lat, junction_lon)
            return (new_lat, new_lon, approach_bearing)
        walked += seg_len
        prev_lon, prev_lat = lon2, lat2

    new_lat = coords[-1][1]
    new_lon = coords[-1][0]
    approach_bearing = bearing_between(
        new_lat, new_lon, junction_lat, junction_lon)
    return (new_lat, new_lon, approach_bearing)
```

Code Listing 3.3. Curve-aware displacement along road geometry

The function walks 75 m along the minor-road edge linestring, accumulating Haversine segment distances. If the road is shorter than the displacement target, the function places the warning at the road end instead. The returned approach bearing records the heading from the displaced warning point toward the junction, which is stored in the mobile dataset and used at runtime for direction matching.

Merge-Focused Danger Score

Listing 3.4 shows the merge-focused danger-score calculation from the Python pipeline.

```
def calculate_enhanced_danger_score(junction_info, parser):
```

```
    road_classes = [
        parser.get_road_classification(edge['highway'])
        for edge in junction_info['edges']
    ]

    if len(road_classes) >= 2:
        class_diff = max(road_classes) - min(road_classes)
        class_score = min(class_diff / 6.0, 1.0)
    else:
        class_score = 0.0

    # angle_score derived from smallest approach-angle pair
    total = (class_score * 0.7) + (angle_score * 0.3)
    return round(min(max(total, 0.0), 1.0), 4)
```

Code Listing 3.4. Merge-focused danger-score calculation

This scoring design intentionally weights class mismatch above angle context because the target scenario is minor-to-major merging. A class difference of six steps maps to a score of 1.0, while angle contribution is capped at 30% of the total.

Listing 3.5 shows how minor roads are selected only when a clear class gap exists.

```
def identify_secondary_roads(junction_info, parser):
    classified = []
    for edge in junction_info['edges']:
        rc = parser.get_road_classification(edge['highway'])
        classified.append({'edge': edge, 'road_class': rc})

    max_class = max(c['road_class'] for c in classified)
    min_class = min(c['road_class'] for c in classified)

    if (max_class - min_class) < MIN_CLASS_DIFFERENCE:
        return []

    return [c for c in classified if c['road_class'] < max_class]
```

Code Listing 3.5. Minor-road selection for merge hazards

This filter is critical for scope control: it avoids generating warnings for junctions where connected roads are of similar hierarchy. The constant `MIN_CLASS_DIFFERENCE`

= 2 defines the minimum road-class gap required to qualify a junction as a minor-to-major merge candidate.

Mobile Export Pipeline

Script `04_export_hazard_data.py` converts the GeoJSON hazard output into a compact JSON format for the mobile app. Listing 3.6 shows the record transformation and compact serialization.

```
def transform_for_mobile(gdf):
    records = []
    for idx, row in gdf.iterrows():
        records.append({
            "id":        int(idx + 1),
            "lat":       round(float(row["warning_lat"]),  6),
            "lon":       round(float(row["warning_lon"]),  6),
            "jLat":      round(float(row["junction_lat"]), 6),
            "jLon":      round(float(row["junction_lon"]), 6),
            "score":     round(float(row["danger_score"]), 3),
            "type":      str(row["junction_type"]),
            "mergeType": str(row.get("merge_type",
                                     "minor_to_major")),
            "bearing":   round(float(row["approach_bearing"]), 1),
            "road":      str(row.get("road_name", "Unnamed")),
            "roadType":  str(row.get("road_type", "unknown")),
            "majorClass":int(row.get("major_road_class", 0)),
            "minorClass":int(row.get("minor_road_class", 0)),
        })
    return records

def save_compact_json(records, output_path):
    with open(output_path, 'w') as f:
        json.dump(records, f, separators=(',', ':'))
```

Code Listing 3.6. Mobile export: record transformation and compact serialization

The export uses six decimal places for coordinates (approximately 0.1 m precision), three decimal places for danger scores, and comma-only separators for compact file size. A separate `generate_sample_subset` function selects the top 50 highest-scoring points for rapid mobile testing.

3.9 Mobile Runtime Implementation

Hazard Service: Spatial Search

Listing 3.7 shows nearby-hazard search with R-tree prefiltering and Haversine confirmation.

```
function findNearby(userLat, userLon, radiusM = PROXIMITY_RADIUS_M) {
  const latPad = metersToLat(radiusM);
  const lonPad = metersToLon(radiusM, userLat);

  let candidates = getAllHazards();
  if (hazardTree) {
    candidates = hazardTree
      .search({
        minX: userLon - lonPad,
        minY: userLat - latPad,
        maxX: userLon + lonPad,
        maxY: userLat + latPad,
      })
      .map((entry) => entry.hazard);
  }

  const nearby = [];
  for (const h of candidates) {
    if (!isValidHazard(h)) continue;
    const dist = haversine(userLat, userLon, h.lat, h.lon);
    if (dist <= radiusM) {
      nearby.push({ ...h, distance: Math.round(dist) });
    }
  }
  nearby.sort((a, b) => a.distance - b.distance);
  return nearby;
}
```

Code Listing 3.7. Nearby search with spatial prefilter and exact distance check

This pattern reduces per-update computation by using the R-tree bounding box as a first-pass candidate filter, then applies exact Haversine evaluation only to the resulting

subset. The output list is sorted by ascending distance so the closest hazard is evaluated first in the warning gate.

Hazard Service: Warning Gate

Listing 3.8 shows the warning decision gate used in runtime checks.

```
function checkForWarning(userLat, userLon, userBearing) {
  if (!isFiniteNumber(userLat) || !isFiniteNumber(userLon)) {
    return null;
  }
  const nearby = findNearby(userLat, userLon, PROXIMITY_RADIUS_M);

  for (const h of nearby) {
    if (h.score < MIN_SCORE_THRESHOLD) continue;
    if (!isCooldownClear(h.id, COOLDOWN_MS)) continue;
    if (!isApproaching(userBearing, h, BEARING_TOLERANCE_DEG))
    ↪ continue;
    return h;
  }
  return null;
}
```

Code Listing 3.8. Runtime warning gate in hazard service

This sequence is the core safety gate that reduces irrelevant notifications. The bearing-check function uses `angleDiff` from `geo.js`: if no bearing is available (e.g., in developer mode with a static fake position), the check defaults to `true`, allowing the warning to fire regardless of heading.

Warning Banner Component

The warning banner is a dedicated React Native component (`src/components/WarningBanner.js`) that displays hazard information when a warning fires. Listing 3.9 shows the component.

```
export default function WarningBanner({ hazard, visible }) {
  if (!visible || !hazard) return null;

  const isHigh     = hazard.score >= 0.7;
  const accentColor = isHigh ? '#cc0000' : '#e68a00';
```

```
  const label       = isHigh ? 'HIGH RISK' : 'CAUTION';

  return (
    <View style={[styles.banner,
                  { borderLeftColor: accentColor }]}>
      <Text style={[styles.label, { color: accentColor }]}>
        {label}
      </Text>
      <Text style={styles.title}>
        {hazard.type} ahead -- {hazard.distance}m
      </Text>
      <Text style={styles.detail}>
        {(hazard.road || 'Unnamed road')
          .replace(/[\[\]']/g, '')}
        {' '}({(hazard.roadType || 'unknown')
          .replace(/[\[\]']/g, '')})
      </Text>
      <Text style={styles.score}>
        Danger score: {hazard.score.toFixed(2)}
      </Text>
    </View>
  );
}
```

Code Listing 3.9. WarningBanner component displaying hazard type, distance, and score

The component applies a red accent for scores at or above 0.7 (HIGH RISK) and orange for lower scores (CAUTION). It displays the junction type, approach distance in metres, road name, road type, and danger score. The banner is rendered in the main `App.js` view and dismissed automatically after eight seconds via a `setTimeout` reference in the parent component.

3.10 Developer Mode Design and Implementation

Developer mode is a deliberately designed in-app testing subsystem that enables repeatable hazard-approach validation without requiring physical movement to hazard locations. This capability was essential during development and for report evidence capture.

Design Rationale

During normal operation, the app relies on live GPS updates. Physical travel to an Oxford junction location during development was impractical. Developer mode addresses this by introducing a simulated GPS layer that can be toggled independently of real location tracking.

Developer mode is accessed via a persistent button in the lower-right corner of the map view. When activated, a test-control panel appears and the button changes state to indicate active developer mode.

Simulated Location Placement

When developer mode is active, the app can compute a simulated GPS position near any loaded hazard. Listing 3.10 shows the `movePointByMeters` function used to generate the simulated position.

```
function movePointByMeters(lat, lon, bearingDeg, distanceM) {
  const latRad     = (lat * Math.PI) / 180;
  const lonRad     = (lon * Math.PI) / 180;
  const bearingRad = (bearingDeg * Math.PI) / 180;
  const angularDistance = distanceM / EARTH_RADIUS_M;

  const newLatRad = Math.asin(
    Math.sin(latRad) * Math.cos(angularDistance) +
    Math.cos(latRad) * Math.sin(angularDistance) *
    Math.cos(bearingRad)
  );
  const newLonRad =
    lonRad +
    Math.atan2(
      Math.sin(bearingRad) * Math.sin(angularDistance) *
      Math.cos(latRad),
      Math.cos(angularDistance) -
      Math.sin(latRad) * Math.sin(newLatRad)
    );

  return {
    latitude:  (newLatRad * 180) / Math.PI,
    longitude: (newLonRad * 180) / Math.PI,
  };
}
```

Code Listing 3.10. movePointByMeters: spherical offset from a known coordinate

This is the same spherical projection formula used in the Python displacement pipeline, applied here in JavaScript to compute the simulated user position.

Random Hazard Simulation

The `setFakeToHazard` function selects a random hazard from the loaded dataset, computes a simulated starting position 20–100 m before the warning point (using the hazard's reverse bearing), and activates fake-GPS mode. Listing 3.11 shows the full implementation.

```
const setFakeToHazard = () => {
  const candidates = hazards
    .filter((h) => Number.isFinite(h.lat) &&
                   Number.isFinite(h.lon))
    .sort((a, b) => b.score - a.score);

  let pool = candidates;
  if (candidates.length > 1 &&
      lastFakeHazardIdRef.current != null) {
    const filtered = candidates.filter(
      (h) => h.id !== lastFakeHazardIdRef.current);
    if (filtered.length > 0) pool = filtered;
  }

  const idx  = Math.floor(Math.random() * pool.length);
  const best = pool[idx];
  lastFakeHazardIdRef.current = best.id;

  // Place simulated user 20-100m before the hazard point
  const randomAheadMeters = 20 + Math.floor(Math.random() * 81);
  const backwardBearing   = Number.isFinite(best.bearing)
    ? (best.bearing + 180) % 360
    : Math.floor(Math.random() * 360);

  const nextFake = movePointByMeters(
    best.lat, best.lon, backwardBearing, randomAheadMeters);
```

```
27
28    setFakeLocation(nextFake);
29    setUseFakeLocation(true);
30    jumpToRegion(makeRegion(
31      nextFake.latitude, nextFake.longitude, 0.02));
32    runWarningCheck(nextFake.latitude, nextFake.longitude, null);
33  };
```

Code Listing 3.11. setFakeToHazard: random hazard simulation for developer mode

The function avoids repeating the same hazard on consecutive presses by maintaining a reference to the last used hazard ID. This supports varied scenario coverage during validation sessions.

State Snapshot and Restoration

Developer mode captures the pre-session map and location state as a snapshot so it can be fully restored. Listing 3.12 shows the snapshot and restoration logic.

```
1  const openDeveloperMode = () => {
2    developerSnapshotRef.current = {
3      useFakeLocation: useFakeLocationRef.current,
4      fakeLocation:    fakeLocationRef.current,
5      region:          mapRegionRef.current || location || null,
6    };
7    if (location) {
8      setDeveloperEntryLocation({
9        latitude:  location.latitude,
10       longitude: location.longitude,
11     });
12   }
13   setDeveloperModeOpen(true);
14 };
15
16 const restoreBeforeDeveloperMode = () => {
17   const snapshot = developerSnapshotRef.current;
18   if (snapshot) {
19     setUseFakeLocation(!!snapshot.useFakeLocation);
20     setFakeLocation(snapshot.fakeLocation ?? null);
21     if (snapshot.region) jumpToRegion(snapshot.region);
```

```
22    }
23    setDeveloperEntryLocation(null);
24    setDeveloperModeOpen(false);
25  };
```

Code Listing 3.12. Developer-mode state snapshot and restoration

This mechanism prevents test-state drift and improves repeatability during validation sessions. When developer mode is opened, the user's real GPS location is captured as a green map marker so it remains visible alongside the simulated position. When the "Back To Normal" button is pressed, the snapshot restores fake-GPS state, map region, and real-location marker to their pre-session values.

Developer Mode Controls Summary

The full set of controls available in developer mode is:

- **Jump Oxford:** animate map to the Oxford city center bounding region.
- **Jump My GPS:** animate map to the user's current real GPS position.
- **Random Fake Hazard:** select a high-scoring hazard at random, compute a simulated approach position, enable fake GPS, and trigger warning check.
- **Use Fake GPS / Use Real GPS:** toggle between simulated and live GPS without changing the current fake-location coordinate.
- **Back To Normal:** restore all state to pre-developer-mode snapshot and close the panel.

3.11 Validation Design

Validation was conducted in two complementary modes:

- **Pipeline validation:** verify that all scripts execute in order and produce expected output files and counts.
- **Runtime validation:** verify warning triggers through live GPS updates and controlled developer-mode simulation.

The runtime warning gate applies four checks in sequence: distance radius ($\leq 300\,\text{m}$), minimum score (≥ 0.35), bearing consistency ($\pm 60^\circ$), and cooldown availability (60 s since last trigger). All four must pass for a warning to fire.

3.12 Interaction Design for In-Motion Use

Because the app is map-first and intended for movement contexts, the interaction design prioritizes concise overlays, minimal taps, and high warning visibility. Controls that are not needed during normal driving are placed in developer mode to avoid clutter. The warning banner is positioned at the bottom of the map, uses large text with a color-coded left border (red for HIGH RISK, orange for CAUTION), and is reinforced by device vibration.

3.13 Verification Strategy

Verification combined script execution checks and runtime behavior checks.

Table 3.3: Verification artifacts and checked behaviors

Artifact	**Primary Focus**	**Checked Behaviors**
Script 01 (`download_osm_data.py`)	Data ingestion	Road graph downloaded or cached, raw files saved correctly.
Script 02 (`detect_junctions.py`)	Junction extraction	Junction counts and type classifications exported correctly.
Script 03 (`generate_hazard_points.py`)	Hazard modeling	Merge hazards and displaced points generated with scoring metadata.
Script 04 (`export_hazard_data.py`)	Mobile export	Compact JSON and sample50 created for app use.
Mobile runtime (`App.js`, `hazardService.js`)	Runtime integration	GPS updates, nearby search, and warning decisions executed correctly.
Developer mode controls	Scenario testing	Simulated location tests validated warning, cooldown, and reset behavior.

End-to-end checks included app startup, permission handling, map rendering, warning trigger behavior, vibration signaling, cooldown behavior, and developer-mode state restoration.

3.14 Reproducibility Procedure

To support independent verification by assessors, the project is reproducible via a short workflow:

1. run Python scripts `01` through `04` in order from project root;
2. verify outputs in `data/raw` and `data/processed`;

3. start the Expo app and connect a physical mobile device via Expo Go;
4. open developer mode, press "Random Fake Hazard" to trigger a simulated warning;
5. validate warning banner, vibration, cooldown, and "Back To Normal" restoration behavior;
6. recompile this report and verify figure, listing, and table cross-references.

3.15 Developer Evidence Capture Plan

For assessor traceability, two implementation-focused screenshots are recommended in addition to runtime screenshots in Appendix A:

- **Developer Evidence D1:** code-editor view of `03_generate_hazard_points.py` showing score and displacement logic.
- **Developer Evidence D2:** code-editor view of `hazardService.js` showing spatial query and warning checks.

3.16 Summary

This chapter presented a methodology that connects open-data geospatial processing to lightweight mobile warning delivery. The implementation is intentionally narrow but complete: it generates hazard data, executes local warning checks, and supports real-device demonstration. Developer mode is a key component that enabled reliable, repeatable validation of the warning pipeline without dependence on physical travel to hazard locations.

The next chapter reports the concrete outputs and measured outcomes from this methodological baseline.

Chapter 4

Results

4.1 Implementation Outcomes

The implemented system satisfies the scoped objectives defined in Chapter 1. It delivers an end-to-end workflow from OSM road-network processing to mobile warning delivery. The resulting artifacts include generated hazard datasets, a functioning Expo mobile application, and reproducible script-based processing steps.

4.2 Feature Completion Summary

The following capabilities were implemented and validated:

- OSM road-network ingestion and junction extraction for Oxford;
- merge-focused hazard scoring and warning-point generation;
- curve-aware displacement of warning points along road geometry;
- compact mobile-data export plus sample test subset generation;
- mobile map runtime with GPS updates and hazard marker rendering;
- warning logic using proximity, score, bearing, and cooldown gates;
- in-app warning banner with HIGH RISK / CAUTION severity levels and vibration feedback;
- developer-mode panel with five distinct test-control actions for repeatable validation.

4.3 Objective Attainment Matrix

Table 4.1 maps the introduction objectives to implemented evidence and attainment status.

Table 4.1: Objective attainment summary

Obj.	Implemented Evidence	Status
O1	Scripted OSM ingestion, junction detection, hazard generation, and export workflow.	Achieved
O2	Minor-to-major merge filtering with class-gap and angle-informed danger scoring.	Achieved
O3	Curve-aware displacement and hazard metadata export for mobile runtime use.	Achieved
O4	React Native app with map rendering, GPS tracking, and hazard-marker display.	Achieved
O5	Real-time warning gate using distance, score, bearing, and cooldown logic.	Achieved
O6	Developer-mode controls for repeatable app-side scenario testing.	Achieved

4.4 Pipeline Output Summary

The current generated artifacts and counts are summarized in Table 4.2.

Table 4.2: Generated dataset outputs from the implemented pipeline

Artifact	Count	Purpose
Junction dataset (`oxford_uk_junctions.geojson`)	2,258	Junction candidates (`street_count` ≥ 3)
Full hazard output (`oxford_uk_hazard_points.geojson`)	455	Hazard points with geometry and metadata
Mobile runtime bundle (`oxford_uk_hazard_mobile.json`)	274	Compact records used by the app
Mobile sample bundle (`oxford_uk_hazard_mobile_sample50.json`)	50	Small test set for rapid validation

The reduction from 2,258 junctions to 455 hazard points reflects the combined effect of the danger-score threshold (minimum 0.35) and the minor-to-major class-gap filter (minimum gap of 2). The further reduction from 455 to 274 compact mobile records is caused by removing duplicate junction entries after scoring and some features with incomplete road metadata.

4.5 Hazard Distribution Snapshot

For the current compact mobile dataset, hazard scores ranged from 0.35 to 0.745 (mean 0.443). Type distribution in the same file was:

- T-junction: 232 records (84.7%)
- Crossroads: 38 records (13.9%)
- 5-way: 4 records (1.5%)

For the full GeoJSON hazard output, 455 features were generated across 415 unique source junctions. The T-junction dominance reflects the Oxford road network topology and the higher frequency of minor-road T-junctions connecting residential streets to primary roads.

4.6 Runtime Configuration and Behavior

Table 4.3 summarizes the runtime warning parameters currently used in the app.

Table 4.3: Runtime warning and location parameters

Parameter	Value	Role
Proximity radius	300 m	Candidate warning distance threshold
Bearing tolerance	60°	Direction match margin for approach checks
Warning cooldown	60 000 ms	Alert de-duplication per hazard ID
Minimum score	0.35	Risk floor before warning can trigger
GPS time interval	10 s	Update cadence for location watcher
GPS distance interval	20 m	Movement threshold for update callbacks
Warning banner timeout	8 000 ms	Auto-dismiss duration for warning banner

The 300 m radius provides comfortable advance warning for a driver approaching at typical urban road speeds. The 60 s cooldown prevents the same hazard from repeating during a slow urban approach. The 60° bearing tolerance provides a balance between direction sensitivity and noise tolerance from GPS drift.

4.7 Developer Mode Validation Results

Developer mode was validated across all five available test-control actions. Table 4.4 summarizes the outcomes from controlled in-app testing.

Table 4.4: Developer mode test control outcomes

Control	Observed Behavior	Result
Jump Oxford	Map animated to Oxford city center region within 700 ms.	Pass
Jump My GPS	Map animated to current real GPS coordinates.	Pass
Random Fake Hazard	Fake position computed 20–100 m before selected hazard; warning check triggered; warning banner displayed; map jumped to simulated region.	Pass
Use Fake GPS / Use Real GPS	Toggle correctly switched between live GPS and fixed simulated position; status overlay updated to reflect active mode.	Pass
Back To Normal	Fake-GPS state cleared; map region restored from snapshot; real-location marker removed from map.	Pass

Multiple "Random Fake Hazard" presses confirmed that the last-used hazard ID exclusion logic works correctly: consecutive presses produced different hazard selections from the candidate pool. The warning banner displayed correct hazard type, approach distance, road name, road type, and danger score on each simulated trigger. Vibration fired in each tested instance.

Cooldown behavior was also confirmed: pressing "Random Fake Hazard" twice in rapid succession for the same hazard correctly suppressed the second warning trigger until the 60-second cooldown elapsed.

4.8 Evaluation Evidence

The repository currently includes a mock telemetry file used for basic analysis checks. Running the analysis script against this file produced:

- location events: 2
- warnings: 3
- true positives: 2
- false positives: 1

- precision: 66.67%

This telemetry sample is suitable for integration validation but insufficient for strong statistical inference. The mock telemetry confirmed that the analysis pipeline executes correctly from raw event files to summary outputs.

4.9 Verification Outcomes by Test Artifact

Table 4.5: Observed outcomes from test artifacts

Artifact	Outcome Summary	Result
Script 01 (`download_osm_data.py`)	Network download and caching completed with raw exports.	Pass
Script 02 (`detect_junctions.py`)	2,258 junctions extracted and classified into T-junction, Crossroads, and 5-way categories.	Pass
Script 03 (`generate_hazard_points.py`)	455 hazard points generated with scores, displacement, and bearing metadata.	Pass
Script 04 (`export_hazard_data.py`)	Compact mobile JSON (274 records) and sample50 generated.	Pass
Mobile startup (Expo + iPhone)	App launched with map, location, and hazard marker rendering.	Pass
Runtime warning gate	Score, bearing, and cooldown conditions applied correctly in all test scenarios.	Pass
Developer mode: all controls	All five controls operated as specified with correct state management.	Pass
Warning banner UI	HIGH RISK / CAUTION severity display, distance, road name, and score rendered correctly.	Pass

4.10 Quality Indicators

Beyond feature completion, three quality indicators are evidenced:

- **Reproducibility:** full pipeline can be rerun from scripts with deterministic output structure.
- **Runtime practicality:** warning checks execute continuously using local data and efficient candidate filtering.
- **Testability:** developer mode enables repeatable hazard-approach scenarios without external tooling or physical travel.

4.11 Screenshot Evidence

All visual evidence is presented in Appendix A. The key evidence map for assessment is:

- Oxford street network (pipeline input): Figure A.1;
- Junction-detection output (2,258 junctions): Figure A.2;
- Displaced hazard warning points (455 points): Figure A.3;
- Warning-point displacement example (75 m offset): Figure A.4;
- Mobile app live map with loaded hazard markers: Figure A.5;
- CAUTION warning banner (T-junction, 45 m, score 0.52): Figure A.6;
- Developer mode test-controls panel: Figure A.7;
- Developer mode with fake GPS active and CAUTION warning: Figure A.8;
- Developer mode with fake GPS active and HIGH RISK warning: Figure A.9;
- Mobile hazard JSON export records: Figure A.10;
- Warning accuracy precision chart: Figure A.11;
- Warning trigger distance distribution: Figure A.12.

4.12 Summary

The implemented system achieves the planned scope and demonstrates reliable end-to-end behavior from data generation to mobile warning delivery. Developer mode was a critical enabler for practical, repeatable validation within project constraints. The evidence supports technical feasibility for the defined scenario and establishes a clear baseline for larger-scale future evaluation.

Chapter 5

Discussion and Analysis

5.1 Interpretation of Findings

The results indicate that a focused, merge-specific warning workflow is technically feasible using open map data and lightweight mobile logic. The implementation integrates three components that are often fragmented in practice: (1) offline geospatial hazard generation, (2) efficient local candidate lookup, and (3) context-aware warning decisions.

The most significant finding is that a narrow scope improved reliability. By targeting minor-to-major merge risk only, the system avoided over-generalized hazard logic and preserved clear decision gates. The reduction from 2,258 candidate junctions to 274 mobile-ready hazard records demonstrates that targeted filtering is effective at concentrating warning energy on the highest-risk subset of the network.

Developer mode proved equally significant in practice. Without it, runtime validation of warning behavior would have required repeated physical travel to Oxford junction locations. The in-app simulation capability made it practical to validate the full warning pipeline — candidate search, bearing check, banner display, cooldown, and state restoration — entirely within a controlled testing session.

5.2 Alignment With Literature

The implementation aligns with the priorities identified in Chapter 2: open-data reproducibility through OSM/OSMnx (OpenStreetMap Foundation, 2026; Boeing, 2017; OSMnx Contributors, 2026), runtime efficiency through spatial indexing (Agafonkin, 2026; Guttman, 1984), and mobile practicality through balanced GPS subscriptions (Expo, 2026).

The project does not claim a new road-safety theory; its contribution is an engineering instantiation of established principles in a constrained but deployable prototype.

5.3 Technical Strengths

Key technical strengths include:

- clear separation between heavy offline processing and lightweight runtime checks;
- explicit hazard metadata design that preserves score, geometry, and approach direction context;
- curve-aware warning-point displacement that places pre-junction alerts on the actual road surface rather than at arbitrary offsets;
- practical runtime warning gate combining distance, score, bearing, and cooldown with no cloud dependency;
- developer-mode subsystem that enables rapid test iteration directly on device, with state snapshot and restoration.

Collectively, these strengths improve maintainability and make system behavior easier to inspect in both code and evidence artifacts. The developer mode design is particularly notable as an engineering practice contribution: building simulated testing capability into the app itself rather than relying on external mocking frameworks made the test workflow stable across device changes and Expo updates.

5.4 Evaluation Against Original Goals (Project Initiation Document)

The original project goals stated in Chapter 1 are evaluated here against implemented outcomes and evidence locations. This makes the assessment linkage explicit rather than implicit.

All core goals in the reduced project scope were met. Broader capabilities, including route guidance, cloud telemetry, and large-scale validation, were intentionally deferred.

5.5 Developer Mode as an Engineering Practice Contribution

Developer mode deserves specific discussion as an engineering pattern rather than just a feature.

The central challenge during mobile safety-app development is the mismatch between the testing environment (a desk, a simulator, or a controlled space) and the deployment

Table 5.1: Evaluation against original project goals

Goal	Target at Initiation	Outcome	Evidence
G1	Build reproducible OSM-to-hazard pipeline	Achieved	Chapter 3, Table 4.2
G2	Focus on minor-to-major merge hazards	Achieved	Listings 3.4 and 3.5
G3	Deliver functional mobile warning app	Achieved	Chapter 4, Figure A.5
G4	Trigger warnings with direction awareness	Achieved	Listing 3.8, Figure A.6
G5	Support practical in-app testing workflow	Achieved	Listings 3.12 and 3.11, Figures A.7 and A.8
G6	Keep implementation scope supervisor-aligned	Achieved	Chapter 1, Section 1.6

environment (a moving vehicle approaching real hazard locations). This mismatch is common to a wide class of location-aware mobile applications and is not solved by standard unit testing.

The developer mode implemented here addresses this by:

1. treating simulated position as a first-class runtime mode, managed through the same state path as live GPS;
2. computing fake positions using the same spherical mathematics as the Python pipeline (function `movePointByMeters`), ensuring geometric consistency;
3. preserving real GPS state in a snapshot so developers can return the app to an unmodified live-GPS state after testing;
4. surfacing a real-location marker on the map when developer mode is open, providing spatial context relative to the simulated position.

This design is reusable as a pattern for other location-aware applications where physical testing at target locations is constrained by time, safety, or logistics.

5.6 Design Trade-offs

Several trade-offs shaped the final design:

- **Scope focus versus feature breadth:** focusing on one hazard scenario improves robustness but limits generality.

- **Offline-first design versus live adaptability:** local data improves reliability and privacy but prevents real-time cloud enrichment.
- **Balanced GPS updates versus granularity:** lower battery impact can miss very short, abrupt movement changes.
- **Simple warning UI versus explanation depth:** concise alerts are practical in motion but provide less post-event explanation.
- **In-app developer mode versus external test harness:** developer mode trades isolation guarantees for ease of access and real-device fidelity.

5.7 Limitations

Although the system meets its defined scope, several limitations remain.

First, evaluation evidence is narrow. The available mock telemetry file is useful for integration checks but too small for robust safety-performance conclusions.

Second, geographic generalization is untested. The pipeline is currently validated on Oxford outputs, and wider reliability across regions with different tagging quality remains future work.

Third, warning relevance can still be affected by GPS drift, especially in dense urban areas. The bearing and cooldown gates reduce this risk but do not remove it.

Fourth, the developer mode bearing check defaults to `true` when no bearing is available (static fake position), meaning simulated warnings fire regardless of simulated heading. This is a deliberate design decision to maximize test coverage of the warning pipeline but means developer mode tests cannot fully exercise the bearing filter.

Fifth, the app is intentionally not a full navigation system. It does not provide route planning or turn-by-turn context, so warning interpretation still depends on user situational awareness.

5.8 Social, Legal, Ethical, and Professional Issues

Social Considerations

The system is intended to improve driving awareness in unfamiliar environments. However, warning systems can create over-reliance if users treat them as a replacement for safe driving judgment. The interface was therefore designed as advisory rather than autonomous control.

Legal Considerations

The current implementation stores hazard data locally and does not require account identity by default, which reduces data-protection exposure. If future versions store route histories or user profiles, UK GDPR compliance requirements become central (Information Commissioner's Office, 2025).

Health and Safety Considerations

This system must not encourage phone interaction while driving. Warnings are designed to be brief and glanceable. Development and testing workflows should still follow safe operation guidance for display-screen use and controlled testing conditions (Health and Safety Executive, 2025).

Ethical Considerations

Ethically, warning systems should avoid false confidence. This report explicitly documents limitations, avoids claims of guaranteed prevention, and treats the app as a decision-support aid rather than a safety guarantee.

5.9 Threats to Validity

Validity considerations are structured as follows:

- **Internal validity:** strengthened by deterministic scripts and explicit runtime warning rules.
- **Construct validity:** moderate, because warning precision from small mock telemetry is only a partial proxy for real-world safety benefit.
- **External validity:** limited, because large-scale field trials and multi-region datasets were outside scope.

Accordingly, claims in this report are technical-feasibility claims rather than broad accident-reduction claims.

5.10 Practical Implications

For software engineering practice, this project provides a repeatable pattern for scenario-focused mobile safety tools: preprocess heavy geospatial logic offline, maintain simple and efficient runtime checks, and provide explicit test controls for rapid validation. The

developer mode architecture in particular represents a practical pattern for any location-aware mobile application where physical testing at target coordinates is impractical.

5.11 Summary

This chapter evaluated the implementation as a practical, constrained engineering artifact. The system is credible for its stated scope, and developer mode is a concrete engineering contribution that improved validation quality within project constraints. Broader evidence, wider deployment, and stronger real-world validation are required before stronger safety claims can be supported. These priorities directly inform Chapter 6.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

This project delivered a complete prototype for merge-focused driving hazard warnings. The final system combines scriptable OSM data processing with a mobile application that performs local, context-aware warning checks in real time.

The key outcome is not broad feature coverage but coherent end-to-end behavior within a constrained scope. Hazard points are generated from road-network structure, exported to mobile-ready format, indexed for efficient runtime lookup, and used to trigger in-app warnings with visual and haptic feedback. A dedicated developer mode enables the full warning pipeline to be validated directly on device without requiring physical travel to hazard locations.

From an engineering perspective, the project demonstrates that meaningful geospatial warning behavior can be achieved with lightweight infrastructure when scope is tightly controlled. The combination of offline hazard generation and on-device spatial search with a multi-gate warning filter provides a stable, testable foundation for a safety-relevant mobile tool.

6.2 Objective Closure

All core objectives in Chapter 1 were met and evidenced in Chapter 4. In summary, the project achieved:

- reproducible OSM-to-hazard data processing scripts running end-to-end without manual correction;
- minor-to-major merge-focused hazard scoring and filtering based on road class gap and approach angle;

- curve-aware warning-point displacement before junction approach, following actual road geometry;
- compact mobile export format and test subset generation;
- mobile runtime warning logic with distance, score, direction, and cooldown gates;
- practical app-side developer controls for repeatable scenario testing including simulated GPS placement, state snapshot, and full restoration.

This objective closure provides a robust baseline for assessment against software quality, implementation rigor, and project-management execution.

6.3 Future Work

Future development should directly address the limitations identified in Section 5.7.

First, multi-region processing should be added to test generalization beyond Oxford. This includes handling regional OSM tagging differences and validating score behavior across varied road hierarchies.

Second, field-trial data collection should be expanded with larger telemetry samples and route-level annotation to evaluate warning precision and recall under realistic driving conditions.

Third, warning logic can be refined through adaptive thresholds based on speed, road context, and repeated false-trigger patterns. Bearing checks could use velocity-derived headings from GPS speed data rather than inferred position deltas.

Fourth, optional cloud-backed analytics can be introduced for aggregated analysis while preserving a privacy-respecting offline mode as default.

Fifth, developer mode testing could be extended to support scripted test sequences, allowing automated regression checks across multiple hazard scenarios in a single session rather than requiring manual button presses.

Finally, the current prototype can be extended with richer user guidance, such as pre-warning explanations and post-event summaries, provided these additions do not increase in-motion distraction.

Taken together, these steps define a practical path from prototype feasibility toward stronger deployment evidence and a more complete safety tool.

Chapter 7

Reflection

This project significantly strengthened my practical software-engineering skills in geospatial data processing and mobile runtime development. Early in the project, I prioritized rapid feature delivery; over time, I learned that reliability and scope discipline are more important than feature volume.

Technical Learning

A major learning point was the sensitivity of geospatial pipelines to data assumptions. Small changes in road-tag handling, geometry availability, or threshold values can materially alter generated hazard outputs. This led me to adopt stricter validation after each script stage and to check output counts explicitly before moving to the next step.

I also improved architectural judgment by separating the offline pipeline from mobile runtime concerns. Keeping heavy processing out of the app reduced complexity and made debugging easier when testing on real devices. Discovering that the displacement algorithm needed to follow road geometry rather than use straight-line offsets was a key design insight: the initial naive approach placed warning points on the wrong side of curves.

Developer Mode as a Design Lesson

One of the most practically valuable lessons came from building developer mode. Initially I planned to test warnings by physically visiting Oxford junction locations. I quickly realized this was impractical and would have severely limited test coverage within project timelines.

Building a simulated GPS layer inside the app turned out to be much more productive than expected. It required designing state-snapshot logic, a fake-location coordinate

system consistent with the real runtime, and UI controls that could not accidentally persist into a live-use session. Getting this right taught me that testability is a first-class design concern, not an afterthought.

The decision to reuse the same spherical displacement formula (`movePointByMeters`) in both the developer mode simulation and in thinking about the pipeline displacement ensured geometric consistency. This consistency would have been harder to achieve if testing had been decoupled from the app codebase.

Mobile and UX Learning

I gained practical experience in map-based mobile interaction design. During testing, I learned that warning interfaces must be concise and highly visible without requiring prolonged reading. The warning banner and vibration pattern were therefore kept simple and immediate. An early version included a dismiss button on the banner; this was removed because it required a tap while driving and added distraction without meaningfully improving the user experience.

Project Management and Evidence Learning

Another key lesson was evidence planning. When I delayed report evidence capture, I had to reconstruct outputs retrospectively. I addressed this by logging key counts, preserving generated files, and maintaining a screenshot plan during implementation. The auto-screenshot macro in the LaTeX template was added specifically to ensure that missing evidence did not block report compilation.

What I Would Do Differently

If repeating the project, I would:

- start multi-region dataset testing earlier instead of focusing on one area first;
- define a larger field-evaluation protocol before final feature tuning, so precision and recall can be computed against real journeys;
- build the developer mode test subsystem earlier, since it significantly accelerated validation quality once in place;
- add more automated regression checks for export consistency and runtime warnings;
- schedule formal report-writing checkpoints earlier in development.

Overall, the project reflects clear progress in planning, implementation discipline, testing rigor, technical reporting, and professional reflection. The developer mode design and the curve-aware displacement algorithm represent the two contributions I am most confident would carry forward into a production-quality iteration of this system.

References

Agafonkin, V. (2026), 'Rbush: High-performance javascript r-tree-based 2d spatial index'. (Accessed: 20 April 2026).
URL: *https://github.com/mourner/rbush*

Boeing, G. (2017), 'Osmnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks', *Computers, Environment and Urban Systems* **65**, 126–139.

Department for Transport (2025), 'Reported road casualties in great britain'. (Accessed: 20 April 2026).
URL: *https://www.gov.uk/government/collections/road-accidents-and-safety-statistics*

Expo (2026), 'Expo location documentation'. (Accessed: 20 April 2026).
URL: *https://docs.expo.dev/versions/latest/sdk/location/*

Google (2026), 'Google maps help'. (Accessed: 20 April 2026).
URL: *https://support.google.com/maps/*

Guttman, A. (1984), R-trees: A dynamic index structure for spatial searching, *in* 'Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data', pp. 47–57.

Health and Safety Executive (2025), 'Working with display screen equipment (dse)'. (Accessed: 20 April 2026).
URL: *https://www.hse.gov.uk/msd/dse/*

Information Commissioner's Office (2025), 'Guide to the uk gdpr'. (Accessed: 20 April 2026).
URL: *https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/*

Meta Open Source (2026), 'React native documentation'. (Accessed: 20 April 2026).
URL: *https://reactnative.dev/docs/getting-started*

OpenStreetMap Foundation (2026), 'Openstreetmap: About'. (Accessed: 20 April 2026).
URL: *https://www.openstreetmap.org/about*

OSMnx Contributors (2026), 'Osmnx documentation'. (Accessed: 20 April 2026).
URL: *https://osmnx.readthedocs.io/*

react-native-maps Maintainers (2026), 'react-native-maps documentation'. (Accessed: 20 April 2026).
URL: *https://github.com/react-native-maps/react-native-maps*

Waze (2026), 'Waze help center'. (Accessed: 20 April 2026).
URL: *https://support.google.com/waze/*

World Health Organization (2023), 'Road traffic injuries'. (Accessed: 20 April 2026).
URL: *https://www.who.int/news-room/fact-sheets/detail/road-traffic-injuries*

Appendix A

System Screenshots

A.1 Pipeline Outputs

Figure A.1: Oxford, UK street network downloaded from OpenStreetMap via OSMnx and used as the input graph for the hazard-detection pipeline.

Figure A.2: All 2,258 road junctions detected in the Oxford network with `street_count` ≥ 3. Each red dot represents one candidate junction node.

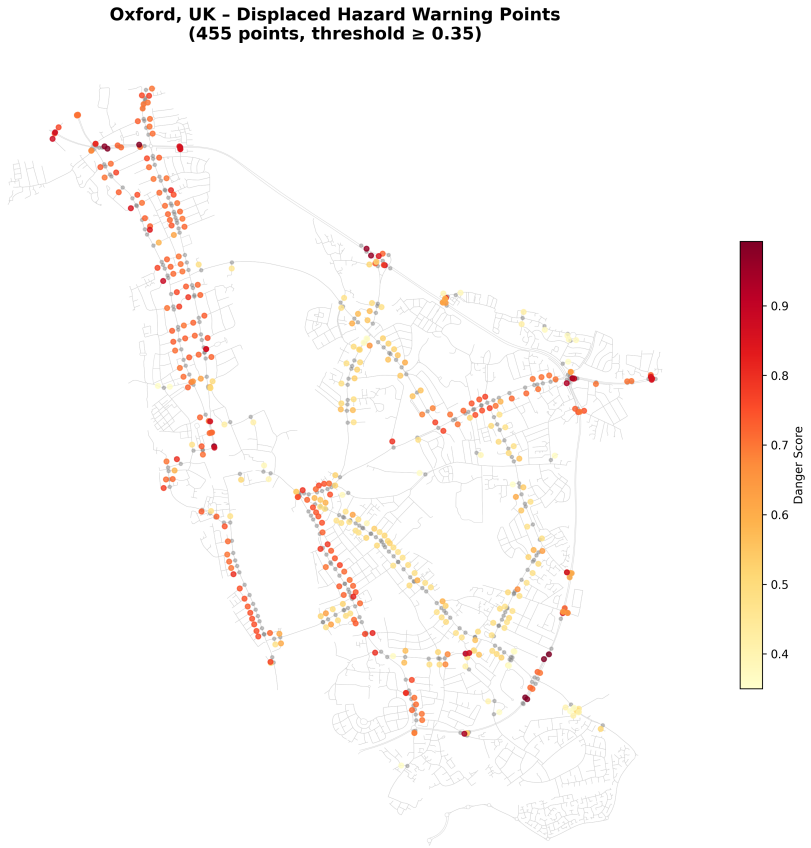

Figure A.3: Displaced hazard warning points generated by the pipeline (455 points, threshold ≥ 0.35). Colour encodes danger score from yellow (low) to dark red (high).

Figure A.4: Displacement example showing junction centres (black dots) and their corresponding displaced warning points (coloured dots) offset 75 m along the road geometry.

A.2 Mobile Application Runtime

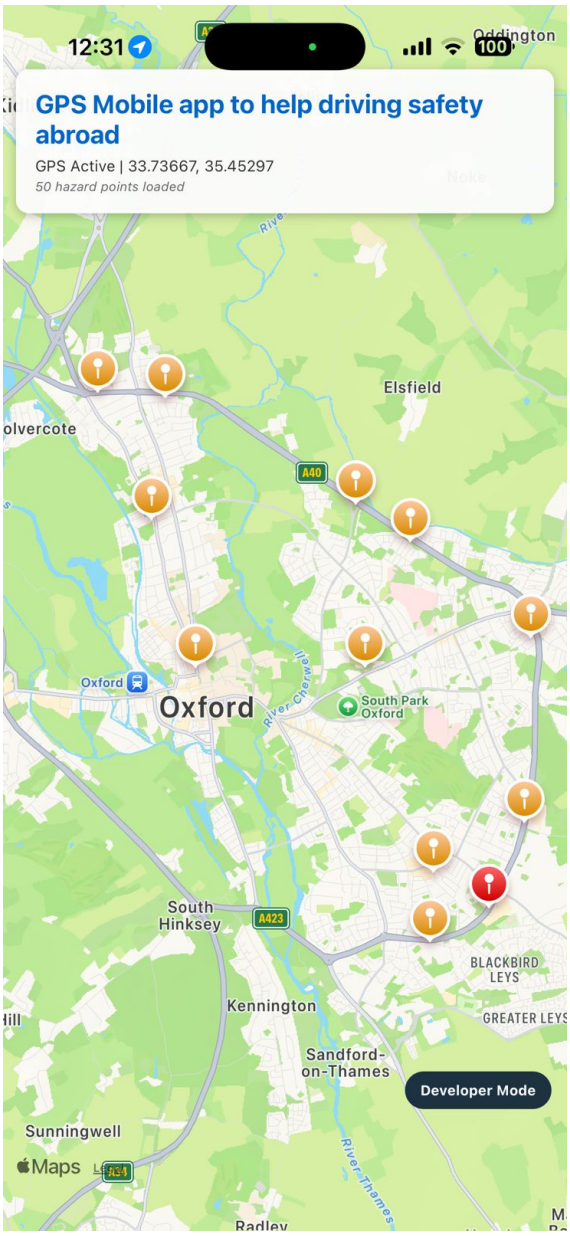

Figure A.5: Mobile app running on iPhone via Expo Go: GPS Active mode with 50 hazard markers loaded. Orange markers indicate CAUTION-level hazards; red markers indicate HIGH RISK hazards.

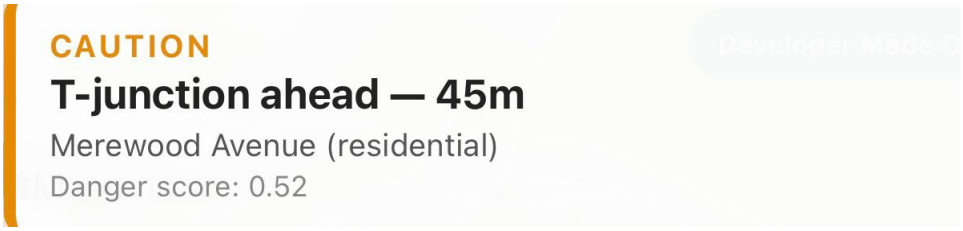

Figure A.6: CAUTION warning banner triggered for a T-junction 45 m ahead on Merewood Avenue (residential road), danger score 0.52.

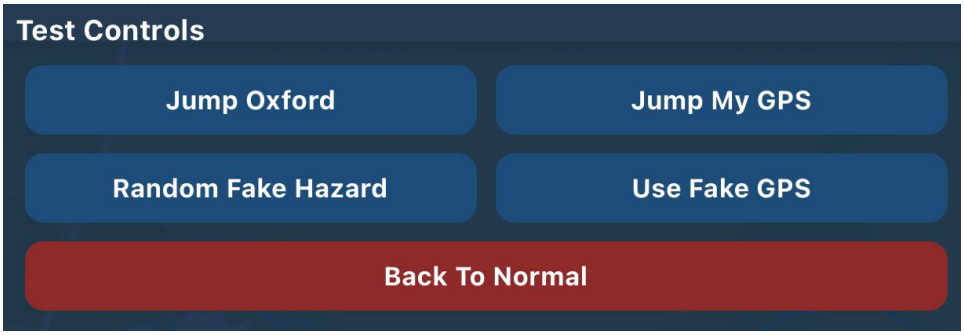

Figure A.7: Developer mode test-controls panel showing all five available actions: Jump Oxford, Jump My GPS, Random Fake Hazard, Use Fake GPS, and Back To Normal.

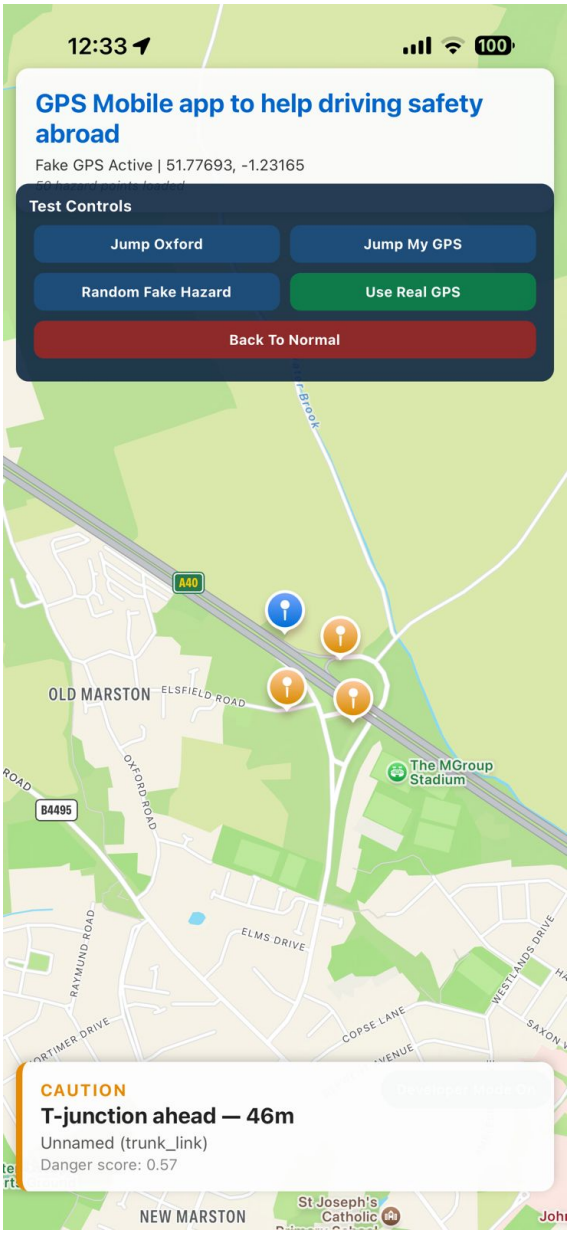

Figure A.8: Developer mode active with Fake GPS enabled (51.77693, −1.23165) and a CAUTION warning triggered: T-junction 46 m ahead, unnamed trunk_link road, danger score 0.57. The blue marker shows the simulated user position.

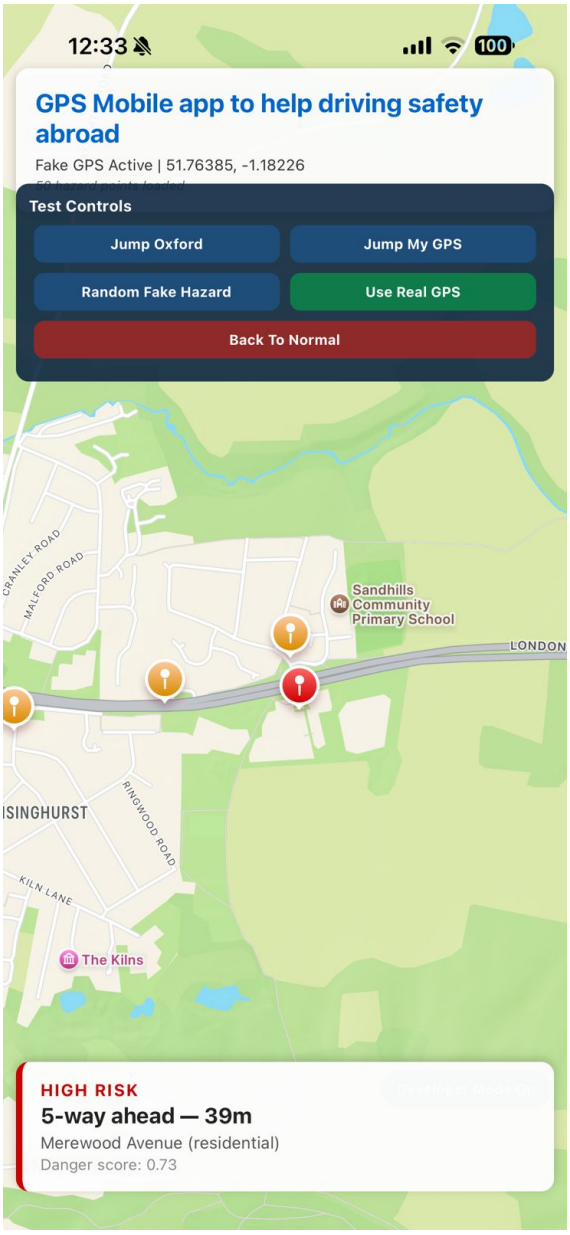

Figure A.9: Developer mode active with Fake GPS enabled (51.76385, −1.18226) and a HIGH RISK warning triggered: 5-way junction 39 m ahead, Merewood Avenue (residential), danger score 0.73. The red marker indicates a high-scoring hazard point.

A.3 Data Export Evidence

```
{
  "id": 1,
  "lat": 51.732075,
  "lon": -1.200864,
  "jLat": 51.732662,
  "jLon": -1.200328,
  "score": 0.745,
  "type": "T-junction",
  "mergeType": "minor_to_major",
  "bearing": 29.5,
  "road": "['Unnamed']",
  "roadType": "['trunk_link']",
  "majorClass": 0,
  "minorClass": 0
},
{
  "id": 2,
  "lat": 51.727939,
  "lon": -1.204319,
  "jLat": 51.727324,
  "jLon": -1.204767,
  "score": 0.743,
  "type": "T-junction",
  "mergeType": "minor_to_major",
  "bearing": 204.3,
  "road": "['Unnamed']",
  "roadType": "['trunk_link']",
  "majorClass": 0,
  "minorClass": 0
},
```


Figure A.10: Compact mobile hazard JSON export showing representative records. Each record contains warning coordinates, junction coordinates, danger score, type, merge type, bearing, road name, road type, and class fields.

A.4 Evaluation Charts

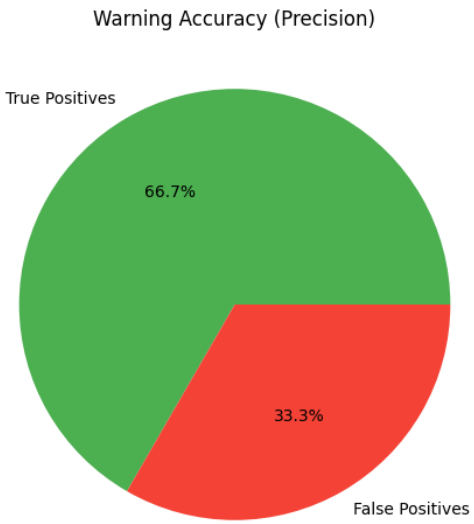

Figure A.11: Warning accuracy (precision) from mock telemetry evaluation: 66.7% true positives and 33.3% false positives from the three warnings generated in the test dataset.

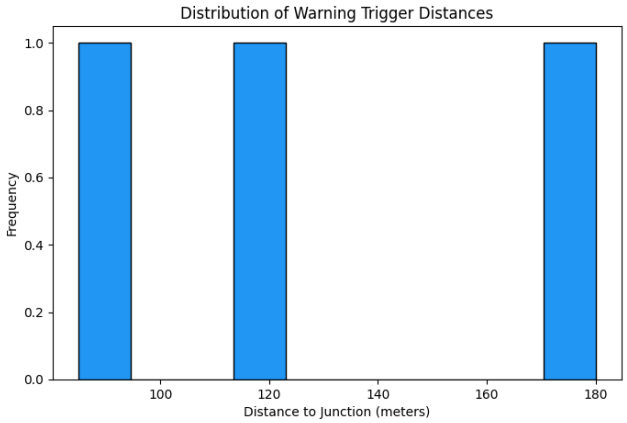

Figure A.12: Distribution of warning trigger distances from the mock telemetry dataset. Warnings were triggered at 90 m, 120 m, and 180 m from the target junction, all within the 300 m proximity radius.

Appendix B

Technical Verification Data

B0. Evidence Alignment Matrix

Table B.1: Where core assessment evidence appears in this report

Criterion	Where Evidence Appears
Problem definition and scope	Chapter 1
Literature grounding	Chapter 2 with citations
Method and architecture quality	Chapter 3 and listings
Implementation and outputs	Chapter 4, Table 4.2
Critical analysis and limitations	Chapter 5
Conclusions and future direction	Chapter 6
Reflective practice	Chapter 7
Screenshot and code evidence	Appendix A and this appendix

B1. Example Mobile Hazard Record


```
{
  "id": 1,
  "lat": 51.753002,
  "lon": -1.255412,
  "jLat": 51.752321,
  "jLon": -1.254801,
  "score": 0.712,
```

```
  "type": "T-junction",
  "mergeType": "minor_to_major",
  "bearing": 146.7,
  "road": "Unnamed",
  "roadType": "residential",
  "majorClass": 8,
  "minorClass": 2
}
```

Code Listing B.1. Representative compact mobile hazard record

B2. Warning-Decision Logic Excerpt

```
function checkForWarning(userLat, userLon, userBearing) {
  const nearby = findNearby(userLat, userLon, PROXIMITY_RADIUS_M);
  for (const h of nearby) {
    if (h.score < MIN_SCORE_THRESHOLD) continue;
    if (!isCooldownClear(h.id, COOLDOWN_MS)) continue;
    if (!isApproaching(userBearing, h, BEARING_TOLERANCE_DEG))
    ↪ continue;
    return h;
  }
  return null;
}
```

Code Listing B.2. Warning gate with score, cooldown, and bearing checks

B3. Reproducibility Command Sequence

1. Run `python src/01_download_osm_data.py`
2. Run `python src/02_detect_junctions.py`
3. Run `python src/03_generate_hazard_points.py`
4. Run `python src/04_export_hazard_data.py`
5. Start mobile app via `npx expo start` from `gps-mobile-safety-abroad/`
6. Open Expo Go on a connected device and scan the QR code

7. Open Developer Mode in the app and validate warning behavior

B4. Examiner Quick-Access Evidence Index

- Project scope and goals: Chapter 1
- Literature and related systems: Chapter 2
- Architecture and implementation method: Chapter 3
- Output datasets and runtime results: Chapter 4
- Critical analysis and limitations: Chapter 5
- Conclusions and future work: Chapter 6
- Pipeline and app screenshot evidence: Appendix A (Figures A.1–A.12)

B5. Code-Snippet Evidence Map

Table B.2: How methodology snippets support assessment claims

Snippet	Assessed Dimension	Evidence Contribution
Listing 3.1	Spatial utility quality	Shows all three geo helpers used throughout runtime: Haversine, bearing, and angle difference.
Listing 3.2	Pipeline correctness	Shows junction extraction and classification from OSM network.
Listing 3.3	Curve-aware design	Shows geometry-following displacement walking road segments.
Listing 3.4	Hazard-model rationale	Shows weighted score focused on class mismatch and angle context.
Listing 3.5	Scope control	Shows filtering to true minor-to-major merge candidates.
Listing 3.6	Export pipeline	Shows mobile-record construction and compact JSON serialization.
Listing 3.7	Runtime efficiency	Shows R-tree prefilter with exact Haversine distance check.
Listing 3.8	Warning correctness gate	Shows score, cooldown, and bearing checks in trigger logic.
Listing 3.9	UI presentation	Shows WarningBanner component with HIGH RISK / CAUTION severity.
Listing 3.10	Developer mode geometry	Shows spherical offset function used for simulated GPS placement.
Listing 3.11	Developer simulation	Shows random hazard selection and fake position computation.
Listing 3.12	Developer validation workflow	Shows snapshot/restore controls for repeatable testing.